

Crearem una estructura de classes i interfícies que ens servirà per simular l'assemblatge de naus que després ens serviran per a la creació d'un joc.

Es crearà un nou projecte amb el nom:
CognomNomExamenUF4

En cas de no respectar el format del nom del projecte, la puntuació de l'examen serà 0 punts.

Les classes seran les següents:

Ammunition
Missile
LaserGenerator

Weapon
MissileLauncher
Laser

Ship
Cruise
BattleShip
ArmoredShip

Propeller
FusionPropellant
IonPropellant

I les interfícies:

ShieldWeakener
HiperSpaceJump

Comencem amb les interfícies:

ShieldWeakener tindrà un únic mètode **weakenShield(Ship s)** que servirà per reduir l'armadura d'una nau enemiga al 50%

HiperSpaceJump tindrà un únic mètode **jumpHiperspace()** que servirà per augmentar la nostra velocitat en un 1000%

Les classes:

Ammunition:

Serà una classe que no podrem instanciar.

Tindrà els atributs **String description; int numberOfShots; int cost;**

El constructor **Ammunition(String description, int numberOfShots, int cost)**

i definirà un mètode **toString()** que mostrarà els atributs d'aquesta classe.

"\n" + descripció + "\n" + "Number Of Shots: " + numberOfShots + "\nCost: " + cost;

Missile

Heretarà d'**Ammunition**

Tindrà el constructor **Missile(String description, int numberOfShots, int cost)**

I el mètode toString de manera que quan la instanciem i la imprimim tinguem la següent sortida per consola:

```
Missile m1 = new Missile("ML-3023 Long Range Missile",200,6000);  
System.out.println(m1);
```

```
ML-3023 Long Range Missile  
Number Of Shots: 200  
Cost: 6000
```

LaserGenerator

Heretarà d'**Ammunition**

Tindrà el constructor **LaserGenerator(String description, int numberOfShots, int cost)**

I el mètode toString de manera que quan la instanciem i la imprimim tinguem la següent sortida per consola:

```
LaserGenerator lg1 = new LaserGenerator("LG3N-A02 Laser Battery",2000,5000);  
System.out.println(lg1);
```

```
LG3N-A02 Laser Battery  
Number Of Shots: 2000  
Cost: 5000
```

Weapon

Serà una classe que no podrem instanciar.

Tindrà els atributs int attackPower; int cost; String description; Ammunition ammunition;

El constructor **Weapon(int attackPower, int cost)**

i definirà un mètode **toString()** que mostrarà els atributs d'aquesta classe.

"\n" + description + "\n"+"Attack Power: "+attackPower+"\nCost: "+cost;

MissileLauncher

Heretarà de **Weapon**

Tindrà el constructor **MissileLauncher(String description,int attackPower, int cost, Missile missile)**

I el mètode toString de manera que quan la instanciem i la imprimim tinguem la següent sortida per consola:

```
MissileLauncher ml1 = new MissileLauncher("Hellstorm Missil Launcher 2",4000,12000,m1);  
System.out.println(ml1);
```

```
Hellstorm Missil Launcher 2  
Attack Power: 4000  
Cost: 12000
```

Observem que m1 ha d'estar creat prèviament.

Laser

Heretarà de **Weapon**

Tindrà el constructor **Laser(String description,int attackPower, int cost, LaserGenerator laserGenerator)**

I el mètode toString de manera que quan la instanciem i la imprimim tinguem la següent sortida per consola:

```
Laser l2 = new Laser("Laser LF-6",800,28000,lg1);  
System.out.println(l2);
```

```
Laser LF-6  
Attack Power: 800  
Cost: 28000
```

Observem que lg1 ha d'estar creat prèviament

Propeller

Serà una classe que no podrem instanciar.

Tindrà els atributs `int power`; `int cost`; `String description`;

El constructor **Propeller(int power, int cost)**

i definirà un mètode **toString()** que mostrarà els atributs d'aquesta classe.

```
"\n"+description+"\n"+"Power: "+power+"\n"+"Cost: "+cost+"\n"
```

FusionPropellant

Heretarà de **Propeller**

Tindrà el constructor **FusionPropellant(String description , int power, int cost)**

I el mètode `toString` de manera que quan la instanciem i la imprimim tinguem la següent sortida per consola:

```
FusionPropellant p1 = new FusionPropellant("G3N-3250",10000, 800 );  
System.out.println(p1);
```

```
Fusion Propellant  
G3N-3250  
Power: 10000  
Cost: 800
```

IonPropellant

Heretarà de **Propeller** .

Implementarà la interfície **HiperSpaceJump**

Tindrà el constructor **IonPropellant(String description , int power, int cost)**

I el mètode `toString` de manera que quan la instanciem i la imprimim tinguem la següent sortida per consola:

```
IonPropellant p2 = new IonPropellant("ZTX-4500",8000,1500);  
System.out.println(p2);
```

```
Ionic Propellant  
ZTX-4500  
Power: 8000 W  
Cost: 1500 U  
Special Feature: Hiper Space Jump
```

Ship

Serà una classe que no podrem instanciar.

Tindrà els atributs `int shield`; `int attackPower`; `int cost`; `int speed`; `Propeller propeller`; `String description`; `Weapon[] weapons`;

El constructor **Ship(int shield, int cost, int speed, int attackPower, Propeller propeller)**

El mètode `int getCost()`, que ens tornarà el cost total de tots els elements assemblats. És a dir, cost de la pròpia nau + cost del propulsor + cost de les armes acoblades + cost de les municions.

El mètode public void `addWeapon(Weapon w)` que assemblarà una arma amb la seva munició. En cas de no tenir espai per acoblar més armes, aquest mètode ens mostrarà el missatge per consola: "Cannot add more weapons"

El mètode public void `removeWeapon(Weapon w)`, al qual li passarem un objecte del tipus `Weapon` i la treurà de la nau. En cas de no tenir acoblat aquest objecte, ens mostrarà el missatge per consola: "This Weapon is not Assembled"

El mètode public void `removeIndexweapon(int i)`, que treurà l'arma amb índex `i` de l'array `weapons`. En cas de no tenir acoblada una arma en aquesta posició de l'array, ens mostrarà el missatge per consola: "This slot has not weapon"

Implementarà la interfície **Comparable** que ens permetrà ordenar els objectes de la classe **Ship** en funció del cost total.

i definirà un mètode **toString()** que mostrarà els atributs d'aquesta classe. Una sortida d'exemple serà:

Armored Ship Phoenix
Speed: 360
Attack Power: 1250

PROPELLER:
Ionic Propellant
TRG-230XT
Power: 15000 W
Cost: 6000 U
Special Feature: Hiper Space Jump

WEAPONS:
Laser LF-6
Attack Power: 800
Cost: 28000
Ammunition:
LSUX-40T Laser Battery
Number Of Shots: 3000
Cost: 8000

Hellstorm Missil Launcher 1
Attack Power: 4000
Cost: 12000
Ammunition:
MS-234F Medium Range Missile
Number Of Shots: 400
Cost: 7000

TOTAL COST: 1461000

Cruise

Heretarà de **Ship** .

Tindrà el constructor **Cruise(String description ,int shield, int cost, int speed,int attackPower ,Propeller propeller)**

L'array weapons només tindrà espai per acoblar 1 arma. Això s'haurà de configurar al constructor

I el mètode toString

```
public String toString() {  
    return "\n====CRUISE====\n" + super.toString();  
}
```

BattleShip

Heretarà de **Ship** .

Tindrà el constructor **BattleShip(String description ,int shield, int cost, int speed,int attackPower , Propeller propeller)**

L'array weapons només tindrà espai per acoblar 3 armes. Això s'haurà de configurar al constructor

I el mètode toString

```
public String toString() {  
    return "\n====BATTLE SHIP====\n" + super.toString();  
}
```

ArmoredShip

Heretarà de **Ship** .

Tindrà el constructor **ArmoredShip(String description ,int shield, int cost, int speed,int attackPower, Propeller propeller)**

L'array weapons només tindrà espai per acoblar 2 armes. Això s'haurà de configurar al constructor
Implementarà la interfície ShieldWeakener, i el mètode weakenShield(Ship s) reduirà la armadura de la nau s a la meitat.

I el mètode toString

```
public String toString() {  
    return "\n====ARMORED SHIP====\n" + super.toString();  
}
```

A la classe principal:

Es crearà un array list de naus `ArrayList<Ship> ships = new ArrayList<Ship>()`, on hi afegirem almenys 6 naus. I mostrarem les naus de l'array list primer ordenades per cost total i després per velocitat. per a aquest darrer cas s'haurà de fer ús de la interfície **Comparator**.

Es deixa com a exemple les instàncies següents perquè puguin ser utilitzades a l'examen.

```
MissileLauncher ml1 = new MissileLauncher("Hellstorm Missil Launcher 1",2000,4000,m1);  
MissileLauncher ml2 = new MissileLauncher("Hellstorm Missil Launcher 2",4000,12000,m2);  
MissileLauncher ml3 = new MissileLauncher("Hellstorm Missil Launcher 2",4000,12000,m1);  
MissileLauncher ml4 = new MissileLauncher("Hellstorm Missil Launcher 1",2000,4000,m2);  
MissileLauncher ml5 = new MissileLauncher("Hellstorm Missil Launcher 1",2000,4000,m1);  
MissileLauncher ml6 = new MissileLauncher("Hellstorm Missil Launcher 1",2000,4000,m2);
```

```
Laser l1 = new Laser("Laser LF-3",600,20000,lg1);  
Laser l2 = new Laser("Laser LF-6",800,28000,lg2);  
Laser l3 = new Laser("Laser LF-6",600,20000,lg1);  
Laser l4 = new Laser("Laser LF-6",800,28000,lg2);
```

```

Laser l5 = new Laser("Laser LF-6",600,20000,lg1);
Laser l6 = new Laser("Laser LF-6",600,20000,lg1);
Laser l7 = new Laser("Laser LF-6",800,28000,lg2);

Cruise c1 = new Cruise("Cruise Yamato Ronin",2000,200000,400,4000,p1);
c1.addWeapon(ml1);
Cruise c2 = new Cruise("Cruise Diminisher",2400,230000,450,4700,p2);
c2.addWeapon(ml2);
BattleShip b1 = new BattleShip("Battle Ship Tartarus",5000, 600000, 250,1500 , p2);
b1.addWeapon(ml3);
b1.addWeapon(ml4);
b1.addWeapon(l1);
BattleShip b2 = new BattleShip("Battle Ship Aegis Elite",5500, 750000, 250 ,1500 , p3);
b1.addWeapon(l2);
b1.addWeapon(l3);
b1.addWeapon(ml5);
ArmoredShip as1 = new ArmoredShip("Armored Ship Phoenix" ,9000, 1400000, 360,1250, p4);
as1.addWeapon(l4);
as1.addWeapon(ml6);
ArmoredShip as2 = new ArmoredShip("Armored Ship Piranha" ,10400, 1600000, 380,1300, p3);
as1.addWeapon(l6);
as1.addWeapon(l7);
ArrayList<Ship> ships = new ArrayList<Ship>();
ships.add(c1);
ships.add(c2);
ships.add(b1);
ships.add(b1);
ships.add(as1);
ships.add(as1);

```

Sistema de Puntuació: La prova tindrà un 10 si no s'aplica cap penalització de les esmentades a continuació:

Atributs no privats **-0,5 punts** a cada classe on es doni el cas.

Formats mostrats per consola no exactes al que es demana **-0,75 punts** a cada cas.

No implementació ordenar per cost total **-1,75 punts**

No implementació ordenar per velocitat **-1,75 punts**

Cada mètode que no funcioni correctament **-0,75 punts**

Classe que hauria de ser abstracta i no ho és **-1 punt**

Herència implementada incorrectament **-1 punt**

No implementar el que es demana a la classe principal **-2 punts**

Cada classe no implementada **-1,5 punts**